SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	20% of CIA	**Total Marks:**	2 * 50 = 100 (1 Question1)
CO3 Statement:	*Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)*		
Student Name:	N.Somashekar Naidu	**Reg. No.:**	192472347
Section / Batch:	CSE-AI	**Date:**	8/10/2026

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

I N.Somashekar Naidu(192472347)*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: N.Somashekar Naidu_____

SECTION A – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

7.Sentiment Analysis Using an LLM API: Develop an application that accepts a user review and determines whether the sentiment is Positive, Negative, or Neutral, with a short explanation.

!.Instructions Relevant to the Implementation

- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and suitable edge cases.
- Use readable, structured, and modular code.
- Add comments and brief documentation.
- Demonstrate the program with suitable test cases.

2. Project Overview

The application accepts a free-text user review and classifies its overall sentiment using the Google Gemini API. A structured prompt instructs the model to return strict JSON containing the sentiment label (Positive, Negative, or Neutral), a confidence score, and a short explanation grounded in the review text. The response is parsed, validated against the three allowed labels, and displayed in the Streamlit interface with a color-coded label, a confidence indicator, and the explanation.

Main Modules

- Review Text Input and Validation
- Prompt Construction (structured JSON instruction)
- Gemini API Call for Sentiment Classification
- JSON Response Parsing and Label Validation
- Result Display (label, confidence, explanation)
- API Error and Edge-Case Handling
- Streamlit User Interface

Technology Stack

Technology	Use
Programming Language	Python
User Interface	Streamlit
Generative AI API	Google Gemini
Configuration	python-dotenv
Response Format	Structured JSON (sentiment, confidence, explanation)

3. Application Workflow

1. User enters a review in the Streamlit text area.
2. Input is validated (empty, too short, too long).
3. A structured prompt is built, instructing Gemini to classify sentiment and return strict JSON.
4. The prompt and review are sent to the Gemini generative model.
5. The JSON response is parsed and the sentiment label is validated against Positive / Negative / Neutral.
6. The result is displayed with a color-coded sentiment label, a confidence bar, and a short explanation.
7. API errors, malformed JSON, and invalid input are all caught and shown as friendly messages instead of crashing the app.

4. Setup and Execution

1. Create/open the project folder in VS Code.
2. Install the required Python packages from requirements.txt.
3. Create a .env file with GEMINI_API_KEY. Keep the API key private; never commit it or share it in screenshots.
4. Run the application using: streamlit run app.py
5. Open the local Streamlit address shown in the terminal.
6. Enter a review and click 'Analyze Sentiment'.

Requirements (requirements.txt):

- streamlit
- google-genai
- python-dotenv

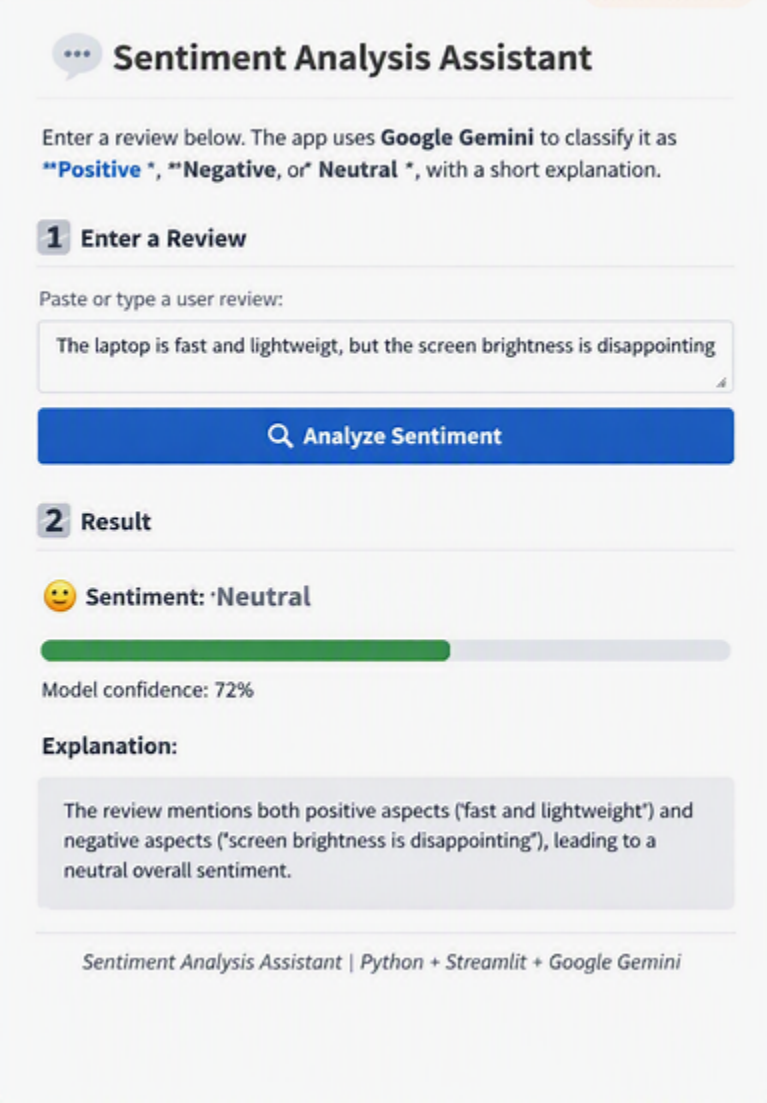
CODE:

```
import os
import json
import re
from pathlib import Path
import streamlit as st
from dotenv import load_dotenv
from google import genai
from google.genai import types

# CONFIGURATION
BASE_DIR = Path(__file__).resolve().parent
ENV_FILE = BASE_DIR / ".env"
load_dotenv(dotenv_path=ENV_FILE)
API_KEY = os.getenv("GEMINI_API_KEY")
GENERATION_MODEL = "gemini-2.5-flash"
VALID_LABELS = {"Positive", "Negative", "Neutral"}

# CLIENT INITIALIZATION
def get_client():
    if not API_KEY:
        raise RuntimeError("Gemini API key not found. Please check .env file.")
    return genai.Client(api_key=API_KEY)

# INPUT VALIDATION
def validate_review(review_text: str) -> str | None:
    if review_text is None or review_text.strip() == "":
        return "Please enter a review before submitting."
    if len(review_text.strip()) < 3:
        return "The review is too short to analyze."
    if len(review_text) > 8000:
```

```
        return "The review is too long (max 8000 characters)."
    return None

# PROMPT CONSTRUCTION
def build_prompt(review_text: str) -> str:
    prompt = f"""
You are a sentiment analysis assistant.
Classify the sentiment of the review below as "Positive", "Negative", or "Neutral".
Respond only in JSON:
{{
  "sentiment": "Positive" | "Negative" | "Neutral",
  "confidence": <float between 0 and 1>,
  "explanation": "<short explanation>"
}}
Review:
\"\"\"
{review_text}
\"\"\"
"""
    return prompt.strip()

# SENTIMENT ANALYSIS (API CALL)
def analyze_sentiment(client: genai.Client, review_text: str) -> dict:
    prompt = build_prompt(review_text)
    response = client.models.generate_content(
        model=GENERATION_MODEL,
        contents=prompt,
        config=types.GenerateContentConfig(temperature=0.2, max_output_tokens=300),
    )
    raw_text = (response.text or "").strip()
    cleaned = re.sub(r"^```(json)?|```$", "", raw_text, flags=re.MULTILINE).strip()
    data = json.loads(cleaned)
    sentiment = data.get("sentiment", "").strip().capitalize()
    confidence = float(data.get("confidence", 0))
    explanation = data.get("explanation", "").strip()
    return {"sentiment": sentiment, "confidence": confidence, "explanation": explanation}

# STREAMLIT USER INTERFACE
def main():
    st.set_page_config(page_title="Sentiment Analysis Assistant", page_icon="    ", layout="centered")
    st.title("    Sentiment Analysis Assistant")
    st.write("Enter a review below. The app uses Google Gemini to classify it as Positive, Negative, or Neutral.")
    st.subheader("1       Enter a Review")
    review_text = st.text_area("Paste or type a user review:", height=160)
    analyze_clicked = st.button("    Analyze Sentiment", type="primary")
```

```
    if analyze_clicked:
        error = validate_review(review_text)
        if error:
            st.warning(error)
            return
        try:
            client = get_client()
            result = analyze_sentiment(client, review_text)
        except Exception as e:
            st.error(f"Error: {e}")
            return

        st.subheader("2       Result")
        sentiment = result["sentiment"]
        color_map = {"Positive": "green", "Negative": "red", "Neutral": "gray"}
        emoji_map = {"Positive": "    ", "Negative": "    ", "Neutral": "    "}
        st.markdown(f"### {emoji_map.get(sentiment, '')} Sentiment: :{color_map.get(sentiment,
'gray')}[{sentiment}]")
        st.progress(result["confidence"])
        st.caption(f"Model confidence: {result['confidence']:.0%}")
        st.markdown("**Explanation:**")
        st.info(result["explanation"])

    st.divider()
    st.caption("Sentiment Analysis Assistant | Python + Streamlit + Google Gemini")

if __name__ == "__main__":
    main()
```

OUTPUT:



36.RAG-Based AI Agent: Develop an AI agent that uses a vector-database retrieval tool to answer questions from a knowledge base. Show the agent → retrieval tool → vector database → LLM workflow.

Instructions Relevant to the Implementation

- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and suitable edge cases.
- Use readable, structured, and modular code.
- Add comments and brief documentation.
- Demonstrate the program with suitable test cases.
- Clearly show the retrieval process and generated response.

2. Project Overview

The application implements an AI agent built on Google Gemini's function-calling capability. Rather than always retrieving context automatically (as a plain RAG pipeline would), the LLM itself is given a retrieval tool and decides when to call it. When the agent calls the tool, it queries an in-memory vector database built from the user-supplied knowledge base — chunks are embedded with Gemini embeddings and ranked by cosine similarity. The retrieved chunks are returned to the agent as the tool's observation, and the agent generates a final answer grounded in that retrieved context.

Main Modules

- Knowledge Base Input and Chunking
- Vector Database (Gemini embeddings + cosine similarity, in-memory)
- Retrieval Tool (function declaration exposed to the agent)
- Agent Loop (tool-call decision -> retrieval -> final answer)
- Workflow Trace Display (agent steps, retrieved chunks, similarity scores)
- API Error and Edge-Case Handling
- Streamlit User Interface

Technology Stack

Technology	Use
Programming Language	Python
User Interface	Streamlit
Generative AI API	Google Gemini (generation, embeddings, function calling)
Agent Mechanism	Gemini native function calling (FunctionDeclaration + Tool)
Vector Database	In-memory NumPy array of Gemini embeddings
Retrieval	Cosine similarity search over embedded knowledge-base chunks

3. Agent / Retrieval Tool / Vector Database / LLM Workflow

The diagram below shows how the four components interact for every question asked to the agent:

```
User Question
     |
     v
  [ AGENT (Gemini) ]  <-- decides whether the retrieval tool is needed
     |
     v   tool call: retrieve_from_knowledge_base(query)
[ RETRIEVAL TOOL ]
```

```
    |
    v   embed query, cosine similarity search
[ VECTOR DATABASE ]  <-- knowledge base chunks + embeddings
    |
    v   top-k relevant chunks returned as the tool's observation
  [ AGENT (Gemini) ]
    |
    v   LLM generates the final answer using retrieved context
Final Grounded Answer
```

4. Step-by-Step Workflow

1. The user pastes knowledge base text into the app.

2. The text is split into overlapping chunks and each chunk is embedded with the Gemini embedding model, forming the vector database.

3. The user asks a question. The question is sent to the agent along with the retrieval tool's definition (name, description, parameters).

4. The agent (LLM) decides whether answering requires the knowledge base. If so, it emits a function call: retrieve_from_knowledge_base(query=...).

5. The app executes the real vector search: the query is embedded, cosine similarity is computed against every chunk, and the top-k chunks are ranked and returned.

6. The retrieved chunks (with similarity scores) are sent back to the agent as the tool's result / observation.

7. The agent generates the final answer using ONLY the retrieved context, and states explicitly if the answer is not available in the knowledge base.

8. The full trace — tool decision, retrieved chunks, similarity scores, and final answer — is displayed in the Streamlit UI.

5. Setup and Execution

1. Create/open the project folder in VS Code.

2. Install the required Python packages from requirements.txt.

3. Create a .env file with GEMINI_API_KEY. Keep the API key private; never commit it or share it in screenshots.

4. Run the application using: streamlit run app.py

5. Open the local Streamlit address shown in the terminal.

6. Paste knowledge base text, click 'Build Vector Database', then ask a question and click 'Run Agent'.

Requirements (requirements.txt):

- streamlit
- google-genai
- numpy
- python-dotenv

6. Implementation and Testing Screenshots

The screenshots below should be captured while running the application locally, in chronological order, so the report documents development, debugging, and final successful demonstrations. Replace each placeholder box with your own screenshot, showing the agent's tool call, the retrieved chunks with similarity scores, and the final generated answer.

CODE:

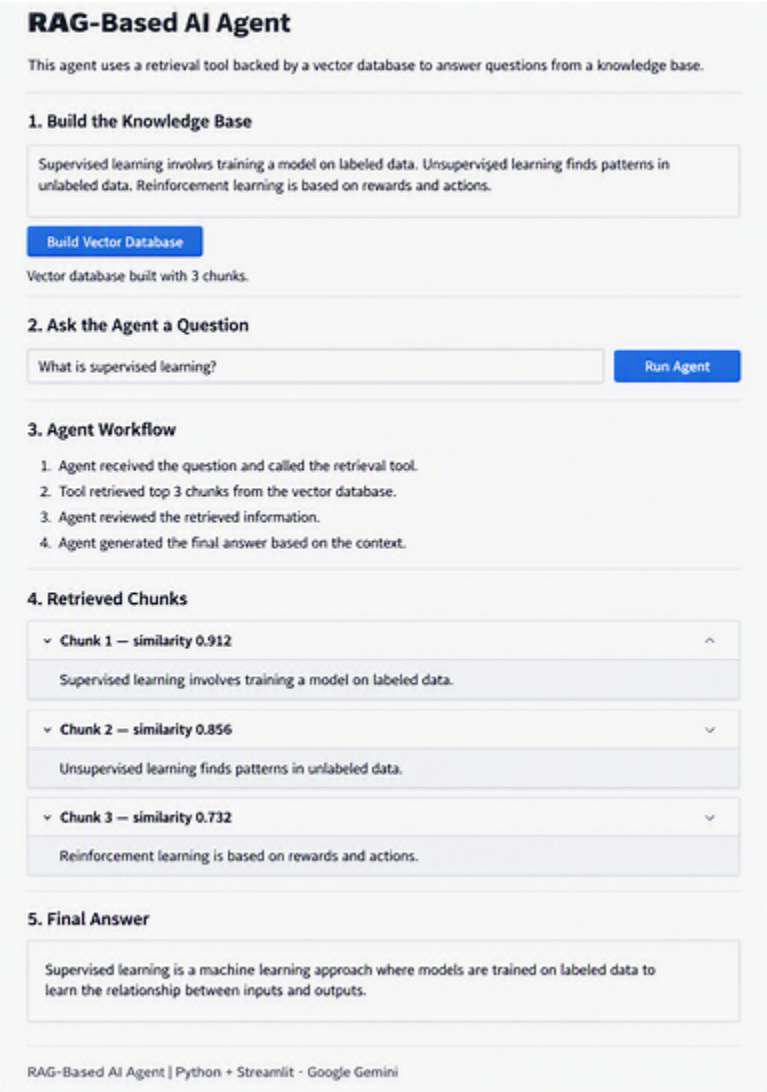
```
import os
import re
import json
from pathlib import Path
import numpy as np
import streamlit as st
from dotenv import load_dotenv
from google import genai
from google.genai import types

# CONFIGURATION
BASE_DIR = Path(__file__).resolve().parent
ENV_FILE = BASE_DIR / ".env"
load_dotenv(dotenv_path=ENV_FILE)
API_KEY = os.getenv("GEMINI_API_KEY")
GENERATION_MODEL = "gemini-2.5-flash"
EMBEDDING_MODEL = "gemini-embedding-001"
CHUNK_SIZE = 500
CHUNK_OVERLAP = 80
TOP_K = 3

# CLIENT INITIALIZATION
def get_client():
    if not API_KEY:
        raise RuntimeError("Gemini API key not found. Please check .env file.")
    return genai.Client(api_key=API_KEY)

# KNOWLEDGE BASE -> CHUNKING
def chunk_text(text: str, chunk_size: int = CHUNK_SIZE, overlap: int = CHUNK_OVERLAP) -> list[str]:
    words = text.split()
    if not words:
        return []
    chunks = []
    start = 0
    while start < len(words):
        end = start + chunk_size
        chunk = " ".join(words[start:end])
        if chunk.strip():
            chunks.append(chunk.strip())
        start += max(chunk_size - overlap, 1)
    return chunks

# VECTOR DATABASE
class VectorDatabase:
    def __init__(self, client: genai.Client):
        self.client = client
        self.chunks: list[str] = []
        self.embeddings: np.ndarray | None = None

    def _embed(self, texts: list[str], task_type: str) -> np.ndarray:
        result = self.client.models.embed_content(
```

```
            model=EMBEDDING_MODEL,
            contents=texts,
            config=types.EmbedContentConfig(task_type=task_type),
        )
        vectors = [np.array(e.values, dtype=np.float32) for e in result.embeddings]
        return np.vstack(vectors)

    def build(self, knowledge_text: str):
        self.chunks = chunk_text(knowledge_text)
        if not self.chunks:
            raise ValueError("The knowledge base is empty after chunking.")
        self.embeddings = self._embed(self.chunks, task_type="RETRIEVAL_DOCUMENT")

    def search(self, query: str, top_k: int = TOP_K) -> list[dict]:
        if self.embeddings is None or len(self.chunks) == 0:
            raise RuntimeError("Vector database is empty. Build it before searching.")
        query_vec = self._embed([query], task_type="RETRIEVAL_QUERY")[0]
        doc_norms = np.linalg.norm(self.embeddings, axis=1)
        query_norm = np.linalg.norm(query_vec)
        denom = doc_norms * query_norm
        denom[denom == 0] = 1e-10
        similarities = (self.embeddings @ query_vec) / denom
        ranked_idx = np.argsort(-similarities)[:top_k]
        return [{"chunk": self.chunks[i], "score": float(similarities[i])} for i in ranked_idx]

# AGENT TOOL DEFINITION
RETRIEVAL_TOOL = types.Tool(
    function_declarations=[
        types.FunctionDeclaration(
            name="retrieve_from_knowledge_base",
            description="Search the vector database and return relevant passages.",
            parameters={
                "type": "OBJECT",
                "properties": {"query": {"type": "STRING"}},
                "required": ["query"],
            },
        )
    ]
)

# AGENT
def run_agent(client: genai.Client, vector_db: VectorDatabase, question: str) -> dict:
    trace = {"steps": [], "retrieved": [], "final_answer": None}
    system_instruction = (
        "You are an AI agent that answers questions using ONLY a knowledge base "
        "accessed through the retrieve_from_knowledge_base tool."
    )
    contents = [types.Content(role="user", parts=[types.Part(text=question)])]
    trace["steps"].append("Agent received the question and was offered the retrieval tool.")
    response = client.models.generate_content(
        model=GENERATION_MODEL,
        contents=contents,
        config=types.GenerateContentConfig(system_instruction=system_instruction, tools=[RETRIEVAL_TOOL],
temperature=0.2),
    )
    candidate = response.candidates[0]
    function_call = None
    for part in candidate.content.parts:
```

```
        if getattr(part, "function_call", None):
            function_call = part.function_call
            break
    if function_call is None:
        trace["steps"].append("Agent answered directly without calling the retrieval tool.")
        trace["final_answer"] = response.text
        return trace
    query = function_call.args.get("query", question)
    trace["steps"].append(f'Agent called tool: retrieve_from_knowledge_base(query=\"{query}\")')
    results = vector_db.search(query, top_k=TOP_K)
    trace["retrieved"] = results
    trace["steps"].append(f'Vector database returned top {len(results)} chunks.')
    tool_result_text = "\n\n".join(f"[Chunk {i+1} | similarity={r['score']:.4f}]\n{r['chunk']}" for i, r in enumerate(results))
    contents.append(candidate.content)
    contents.append(types.Content(role="user",
parts=[types.Part.from_function_response(name="retrieve_from_knowledge_base", response={"result":
tool_result_text})]))
    trace["steps"].append("Retrieved chunks were returned to the agent.")
    final_response = client.models.generate_content(
        model=GENERATION_MODEL,
        contents=contents,
        config=types.GenerateContentConfig(system_instruction=system_instruction, tools=[RETRIEVAL_TOOL],
temperature=0.2),
    )
    trace["steps"].append("Agent generated the final grounded answer.")
    trace["final_answer"] = final_response.text
    return trace

# INPUT VALIDATION
def validate_knowledge_base(text: str) -> str | None:
    if not text or not text.strip():
        return "Please paste or upload some knowledge base text first."
    if len(text.strip()) < 50:
        return "The knowledge base is too short."
    return None

def validate_question(text: str) -> str | None:
    if not text or not text.strip():
        return "Please enter a question."
    if len(text.strip()) < 3:
        return "The question is too short."
    return None

# STREAMLIT USER INTERFACE
def main():
    st.set_page_config(page_title="RAG-Based AI Agent", page_icon="    ", layout="centered")
    st.title("    RAG-Based AI Agent")
    st.write("This agent uses a retrieval tool backed by a vector database to answer questions from a knowledge base.")
    if "vector_db" not in st.session_state:
        st.session_state.vector_db = None
        st.session_state.kb_chunks_count = 0
    st.subheader("1       Build the Knowledge Base")
    knowledge_text = st.text_area("Paste knowledge base text:", height=200)
    if st.button("    Build Vector Database"):
        error = validate_knowledge_base(knowledge_text)
        if error:
            st.warning(error)
        else:
```

```
            try:
                client = get_client()
                with st.spinner("Chunking text and generating embeddings..."):
                    vdb = VectorDatabase(client)
                    vdb.build(knowledge_text)
                    st.session_state.vector_db = vdb
                    st.session_state.kb_chunks_count = len(vdb.chunks)
                st.success(f"Vector database built with {len(vdb.chunks)} chunks.")
            except Exception as e:
                st.error(f"Error: {e}")
    if st.session_state.vector_db is not None:
        st.caption(f"    Knowledge base ready — {st.session_state.kb_chunks_count} chunks embedded.")
        st.subheader("2       Ask the Agent a Question")
        question = st.text_input("Enter your question:")
        if st.button("    Run Agent", type="primary"):
            q_error = validate_question(question)
            if q_error:
                st.warning(q_error)
            else:
                try:
                    client = get_client()
                    with st.spinner("Agent is working..."):
                        trace = run_agent(client, st.session_state.vector_db, question)
                except Exception as e:
                    st.error(f"Error: {e}")
                    return
                st.subheader("3       Agent Workflow")
                for i, step in enumerate(trace["steps"], start=1):
                    st.markdown(f"**Step {i}:** {step}")
                if trace["retrieved"]:
                    st.subheader("4       Retrieved Chunks")
                    for i, r in enumerate(trace["retrieved"], start=1):
                        with st.expander(f"Chunk {i} — similarity {r['score']:.4f}"):
                            st.write(r["chunk"])
                st.subheader("5       Final Answer")
                st.info(trace["final_answer"] or "No answer was generated.")
    st.divider()
    st.caption("RAG-Based AI Agent | Python + Streamlit + Google Gemini")

if __name__ == "__main__":
    main()
```

OUTPUT:



Final Demonstration Results

Test	Input	Expected Output	Status
Test Case 1	Relevant question about the pasted knowledge base	Agent calls the retrieval tool, retrieves relevant chunks, and answers grounded in them	Pending demo
Test Case 2	Question with a very specific fact stated in the knowledge base	Agent retrieves the correct chunk and answers accurately	Pending demo
Edge Case 1	Question unrelated to the knowledge base	Agent states the answer is not available in the knowledge base	Pending demo
Edge Case 2	Question asked before building the vector database	App prompts the user to build the knowledge base first	Pending demo
Edge Case 3	Missing/invalid API key	Friendly configuration error message	Pending demo

Update the "Status" column to "Successful" once you have run each test case against the live application, and attach the corresponding screenshot in Section 6.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
API Integration Correctness	30%	Correct, efficient API integration; understands request/response flow	Correct integration with minor inefficiencies	Partial integration with functional errors	API integration non-functional / major errors
Problem-Solving Completeness	25%	Solves the given problem completely and robustly	Solves problem with minor issues	Partial solution to the problem	Does not solve the assigned problem
Code Quality	20%	Clean, well-structured, error-handled code	Reasonably clean code	Code works but poorly structured	Code is unreliable or unstructured
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases	Demo covers the main use cases	Demo is incomplete	No functional demo presented
Code Documentation	10%	Well-commented code with clear README/setup steps	Adequate comments/instructions	Minimal documentation provided	No documentation provided
Code Quality	20%	Clean, well-structured, error-handled code	Reasonably clean code	Code works but poorly structured	Code is unreliable or unstructured
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases	Demo covers the main use cases	Demo is incomplete	No functional demo presented
Code Documentation	10%	Well-commented code with clear README/setup steps	Adequate comments/instructions	Minimal documentation provided	No documentation provided

Marks Evaluation:

Question No.	API Integration Correctness(30 %)	Problem-Solving Completeness (25%)	Code Quality(20 %)	Input/Output Demo(15%)	Code Documentation (10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	